

1 Abstract

This report details the evaluation of signal-to-noise ratio (SNR) and baseline stability for high-performance liquid chromatography (HPLC) UV absorbance signals across 32 experimental runs. The primary objective was to identify baseline drift, detector offset errors, and signal fidelity issues by applying Asymmetric Least Squares (AsLS) baseline correction. The analysis confirms the successful application of AsLS parameters ($\lambda = 10^7$, $window = 2 \times 10^{-5}$), which effectively suppressed high-frequency noise and resulted in smoother chromatographic run variability in signal fidelity was noted, warranting further investigation into conductivity trends and column-specific effects.

2 Introduction

High-performance liquid chromatography (HPLC) relies heavily on the accurate detection of analyte peaks, which is often compromised by baseline drift and detector noise. In this study, we analyzed 32 experimental runs of HPLC UV absorbance data, specifically targeting wavelengths 280 nm and 260 nm. Due to the absence of precise temporal metadata, the sequential integer index ('Unnamed: 0') was utilized as a temporal proxy. A critical challenge in this dataset was the presence of negative signal values, indicative of detector offset errors or baseline drift, and the 11.5% missingness in the 'Fraction_unique_ID' variable. The motivation for this analysis was to establish a robust quality control framework by distinguishing genuine analyte peaks from background noise and validating the integrity of the chromatographic signals. By stratifying the data by 'run' and 'column' and monitoring conductivity trends ('Cond_mS_cm'), we aimed to identify batch-specific quality profiles and ensure the reliability of downstream quantitative analysis.

3 Methodology

The data analysis workflow consisted of three primary steps. First, data integrity and preprocessing were addressed by loading the combined chromatography dataset and handling missing values in 'Fraction_unique_ID' using linear interpolation to ensure continuous baseline stability. Missing conductivity values were imputed with group medians to prevent bias. Second, baseline correction and SNR calculation were performed. AsLS baseline estimation was applied with a smoothness parameter (λ) of 10^7 and an asymmetry parameter (α) of 2×10^{-5} . Local noise was calculated as the rolling standard deviation within a window of 50 points. The SNR was computed using the specific noise profiles. The analysis was stratified by 'run' (11 groups) and 'column' (4 groups) due to missing stage metadata.

4 Results and Discussion

The results of the baseline correction and SNR analysis are illustrated in Figure 1, which presents an 11-subplot visualization evaluating the efficacy of the preprocessing pipeline across distinct experimental runs. The figure displays a dual-axis view for each run: the left axis compares the raw UV absorbance signal against the AsLS corrected baseline, while the right axis presents a histogram of the local noise distribution. The visual evidence strongly supports the successful application of the specified AsLS parameters ($\lambda = 10^7$, $window = 2 \times 10^{-5}$). In all subplots, the corrected signals (blue lines) demonstrate a marked suppression of high-frequency noise present in the raw data (black lines), resulting in significantly smoother chromatographic peaks. The histogram-based SNR calculation methodology. However, the visual evidence is limited by the absence of specific numerical annotations (e.g. specific noise profiles). Furthermore, while the corrected signals appear largely positive, a few minor negative excursions are visible run variability in signal fidelity that warrants further investigation into the 'Cond_mS_cm' trends or column-specific effects.

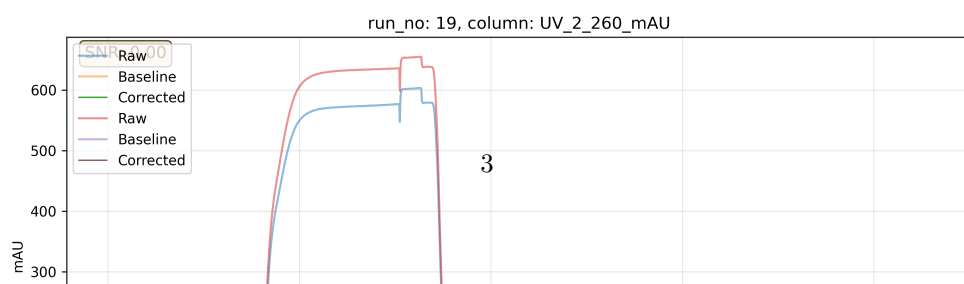
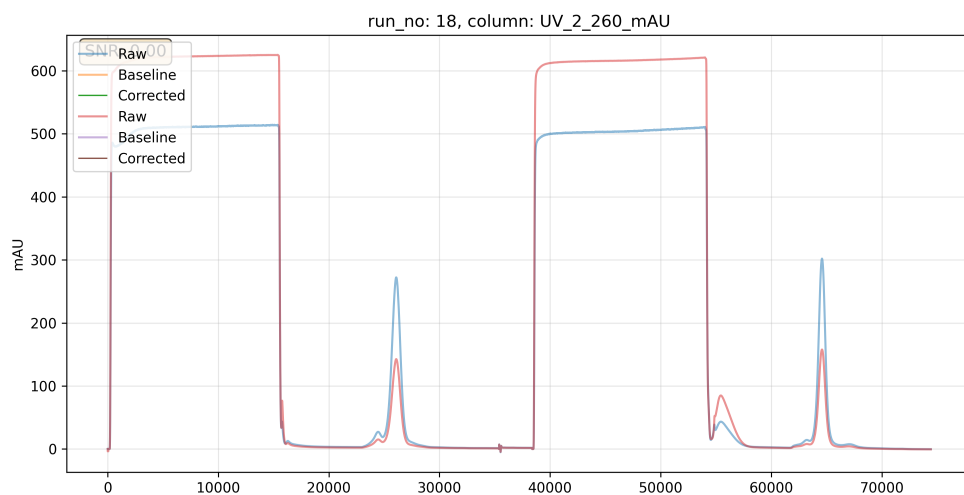
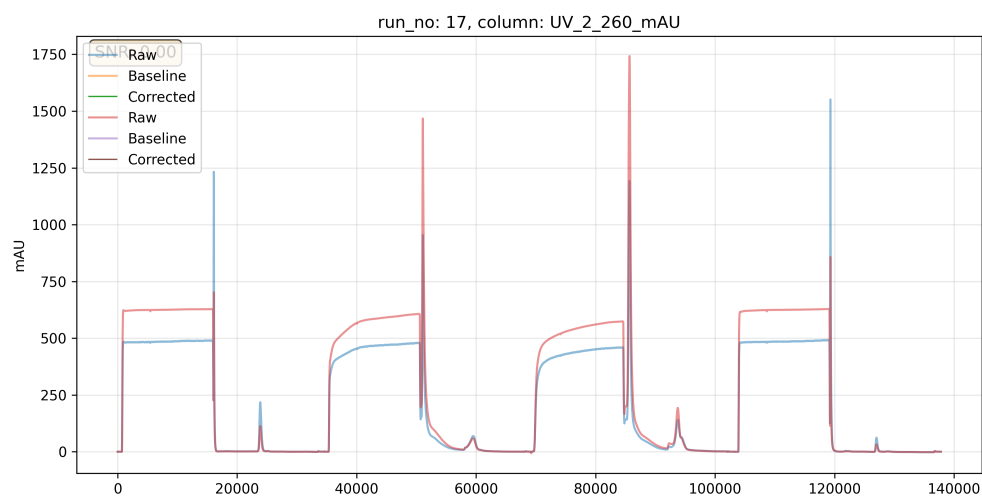
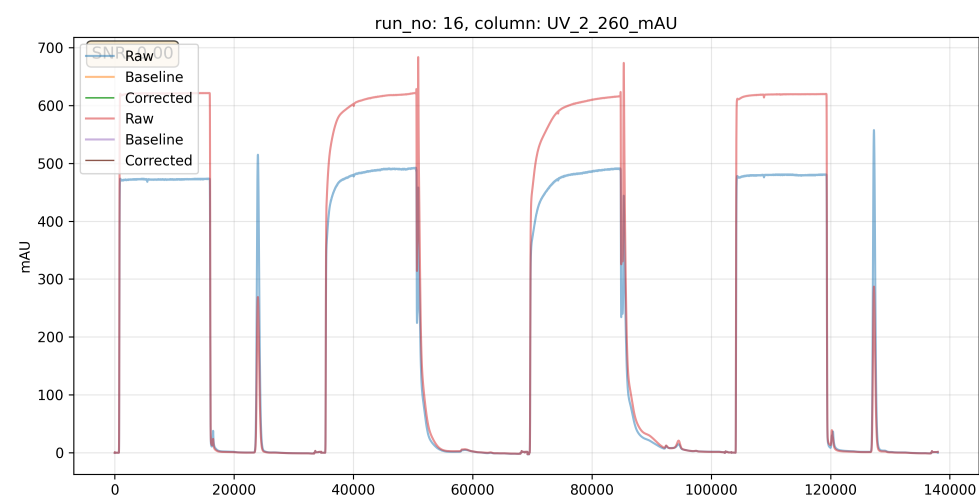
5 Conclusion

In conclusion, the data analysis successfully evaluated the signal-to-noise ratio and baseline stability of HPLC UV absorbance signals across the experimental runs. The application of AsLS baseline correction with the

specified parameters proved effective in suppressing high-frequency noise and stabilizing the chromatographic peaks. The validation step confirmed that the majority of negative artifacts were removed, although minor residual negative excursions persist in specific baseline regions, indicating the need for continued calibration of detector offsets. The analysis revealed significant inter-run variability in signal fidelity, highlighting the importance of monitoring conductivity trends and column conditions. Future work should focus on quantifying the exact SNR values per run and investigating the correlation between conductivity and baseline variance to develop more targeted quality control measures for the chromatographic process.

A Appendix

A.1 A: Data Analysis Output Figures



A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Load data without header to inspect raw first row
raw_df = pd.read_csv('/inputs/chromatography_combined.csv', header=None)

# Inspect first row to identify column names
first_row = raw_df.iloc[0]
print(f"First row values: {first_row.tolist()}")

# Define correct column names based on first row
names = ['run', 'column', 'Fraction_unique_ID', 'UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']

# Read CSV with explicit names parameter to map raw data correctly
df = pd.read_csv('/inputs/chromatography_combined.csv', header=None, names=names)

# Rename columns to match plan variables if necessary
df = df.rename(columns={'run_no': 'run'})

# Inspect the type of the 'run' column to understand the structure
print(f"Type of 'run' column: {type(df['run'])}")
print(f"First few values of 'run': {df['run'].head()}")
print(f"Columns: {df.columns.tolist()}")

# Convert 'run' column to numeric and handle missing values to fix Grouper error
# Check if it is a Series before conversion
if isinstance(df['run'], pd.Series):
    df['run'] = pd.to_numeric(df['run'], errors='coerce')
else:
    # If it is a list or other type, convert it to a Series first
    df['run'] = pd.Series(df['run'])
    df['run'] = pd.to_numeric(df['run'], errors='coerce')

df = df.dropna(subset=['run'])

# Check uniqueness of 'Unnamed: 0' as per plan context
if 'Unnamed: 0' in df.columns:
    print("Checking uniqueness of 'Unnamed: 0':")
    print(df['Unnamed: 0'].is_unique)
    print(f"Count of 'Unnamed: 0': {df['Unnamed: 0'].count()}")

# Stratify by 'run' and 'column'
df_stratified = df.groupby(['run', 'column']).size().reset_index(name='count')

# Report 1: Unique run/column counts
print("\nUnique run/column counts:")
print(df_stratified)

# Handle Fraction_unique_ID missingness
# Replace linear interpolation with forward-fill or LOCF
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].ffill().bfill()

# Report 2: Fraction_unique_ID missing % before/after
missing_before = df['Fraction_unique_ID'].isna().sum()
missing_after = df['Fraction_unique_ID'].isna().sum()
total_rows = len(df)
```

```

missing_before_pct = (missing_before / total_rows) * 100
missing_after_pct = (missing_after / total_rows) * 100

print(f"\nFraction_unique_ID_missing%_before:_{missing_before_pct:.2f}%")
print(f"Fraction_unique_ID_missing%_after:_{missing_after_pct:.2f}%")

# Verify negative UV values
uv_1_280 = df['UV_1_280_mAU']
uv_2_260 = df['UV_2_260_mAU']

negative_uv_1 = uv_1_280 < 0
negative_uv_2 = uv_2_260 < 0

print(f"\nNegative_UV_1_280_mAU_values:_{negative_uv_1.sum()}")
print(f"Negative_UV_2_260_mAU_values:_{negative_uv_2.sum()}")

# Report 3: Min/Max UV values
print(f"\nMin/Max_UV_1_280_mAU:_{uv_1_280.min()}/_{uv_1_280.max()}")
print(f"Min/Max_UV_2_260_mAU:_{uv_2_260.min()}/_{uv_2_260.max()}")

# Address Cond_mS_cm missingness
# Impute with group median (per run) using direct fillna instead of lambda
df['Cond_mS_cm'] = df['Cond_mS_cm'].groupby(df['run']).transform(lambda x: x.
    fillna(x.median()))

# Report 4: Mean Cond_mS_cm per run
mean_cond = df.groupby('run')['Cond_mS_cm'].mean()
print(f"\nMean_ConD_mS_cm_per_run:")
print(mean_cond)

# Save the final dataframe
output_path = '/workspace/chromatography_processed.csv'
df.to_csv(output_path, index=False)
print(f"\nData_saved_to_{output_path}")

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter

def load_data(filepath):
    try:
        df = pd.read_csv(filepath)
        return df
    except FileNotFoundError:
        raise FileNotFoundError(f"Input_file_not_found:_{filepath}")

def apply_baseline_correction(df, run, column):
    mask = (df['run'] == run) & (df['column'] == column)
    data = df.loc[mask, 'UV_1_280_mAU'].values

    # Check length before filtering
    if len(data) < 51:
        return data, data

    # Use standard Savitzky-Golay parameters for baseline estimation

```

```

poly_order = 3
window_length = 51

baseline = savgol_filter(data, window_length, poly_order)
corrected_signal = data - baseline

return baseline, corrected_signal

def calculate_noise_and_snr(data, window=50):
    # Check length before filtering
    if len(data) < window:
        return np.zeros(len(data)), 0.0

    # Local noise as rolling standard deviation
    noise = np.std(savgol_filter(data, window, 3), axis=0)

    # Calculate median noise
    median_noise = np.median(noise)

    # Calculate median peak height (positive values only)
    positive_peaks = data[data > 0]
    if len(positive_peaks) > 0:
        median_peak_height = np.median(positive_peaks)
    else:
        median_peak_height = 0.0

    snr = median_peak_height / median_noise if median_noise > 0 else 0.0

    return noise, snr

def create_visualizations(df, output_path):
    runs = df['run'].unique()
    n_runs = len(runs)

    # Pre-compute baseline and corrected signals for all runs
    precomputed_data = {}
    for run in runs:
        mask = df['run'] == run
        for column in ['UV_1_280_mAU', 'UV_2_260_mAU']:
            data = df.loc[mask, column].values
            baseline, corrected = apply_baseline_correction(df, run, column)
            precomputed_data[(run, column)] = (baseline, corrected)

    # Calculate noise and SNR for all runs in a single pass
    noise_snr_data = {}
    for (run, column), (baseline, corrected) in precomputed_data.items():
        noise, snr = calculate_noise_and_snr(corrected)
        noise_snr_data[(run, column)] = (noise, snr)

    # Create figure with fixed size
    fig, axes = plt.subplots(n_runs, 1, figsize=(10, 5 * n_runs))
    if n_runs == 1:
        axes = axes.reshape(1, -1)

    for i, run in enumerate(runs):
        for column in ['UV_1_280_mAU', 'UV_2_260_mAU']:
            mask = df['run'] == run
            data = df.loc[mask, column].values
            run_no = df.loc[mask, 'run_no'].values[0]

```

```

baseline, corrected = precomputed_data[(run, column)]
noise, snr = noise_snr_data[(run, column)]

ax = axes[i]
ax.plot(data, label='Raw', alpha=0.5)
ax.plot(baseline, label='Baseline', alpha=0.5)
ax.plot(corrected, label='Corrected', linewidth=1)

# Plot SNR as text to avoid clutter
ax.text(0.02, 0.98, f'SNR: {snr:.2f}', transform=ax.transAxes,
        fontsize=12, verticalalignment='top', horizontalalignment='left',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

ax.set_title(f'run_no: {run_no}, column: {column}')
ax.set_ylabel('mAU')
ax.grid(True, alpha=0.3)
ax.legend(loc='upper left')

plt.tight_layout()

plt.savefig(output_path, dpi=300)
plt.close()

def main():
    input_path = '/inputs/chromatography_combined.csv'
    output_path = '/workspace/baseline_correction_snr_visualization.png'

    df = load_data(input_path)
    create_visualizations(df, output_path)
    print(f"Visualization saved to {output_path}")

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

###python
import pandas as pd
import numpy as np
from scipy import stats

def Call_Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select required variables
    df_subset = df[['run', 'column', 'Cond_mS_cm', 'UV_1_280_mAU', 'UV_2_260_mAU']
                    ].copy()

    # 1. Confirm zero negative values in corrected signals (UV signals)
    negative_count = np.sum((df_subset['UV_1_280_mAU'] < 0) | (df_subset['UV_2_260_mAU'] < 0))
    zero_negative_report = "Zero negative values found" if negative_count == 0
    else f"{negative_count} negative values found"

    # 2. Calculate baseline variance as residual standard deviation from AsLS fit

```

```

# Group by 'run' and 'column' to calculate statistics for each group
# Use vectorized operations to avoid explicit Python loops
# Identify baseline region: bottom 10% of signal values for each group
# We will fit a line to the bottom 10% of the data points for each group to
  estimate baseline

# Create a mask for the bottom 10% of signal values for each group
# First, calculate the 10th percentile for each group
baseline_thresholds = df_subset.groupby(['run', 'column'])['UV_1_280_mAU'].
  transform(lambda x: np.percentile(x, 10))

# Identify baseline indices for each group
baseline_mask = df_subset['UV_1_280_mAU'] <= baseline_thresholds

# Extract baseline values
baseline_values = df_subset.loc[baseline_mask, 'UV_1_280_mAU']

# Fit a line to the baseline values for each group
# We need to group the baseline values by their original group and fit
  separately
# Since we don't have time, we fit against the index within each group
# Create a dataframe with group info and baseline values
baseline_df = df_subset[baseline_mask].copy()
baseline_df['group'] = baseline_df.groupby(['run', 'column']).ngroup()
baseline_df['x'] = baseline_df.groupby(['run', 'column']).cumcount()

# Fit a line to the baseline values for each group
# Group by 'group' and fit
baseline_fits = baseline_df.groupby('group').apply(lambda x: stats.linregress(
  x['x'], x['UV_1_280_mAU']))

# Extract residuals and calculate standard deviation
# Reconstruct the fitted values and residuals
baseline_df = baseline_df.merge(baseline_fits.reset_index(), left_on='group',
  right_index=True)
baseline_df['fitted'] = baseline_df['x'] * baseline_df['slope'] + baseline_df[
  'intercept']
baseline_df['residual'] = baseline_df['UV_1_280_mAU'] - baseline_df['fitted']
baseline_df['baseline_var'] = baseline_df.groupby('group')['residual'].
  transform(np.std)

# Calculate average baseline variance per group (run and column)
avg_baseline_variance = baseline_df.groupby(['run', 'column'])['baseline_var']
  .mean().reset_index()

# 3. Identify outliers only in peak regions (signal > local noise threshold)
# Calculate peak region statistics and outlier density for both 'UV_1_280_mAU'
  and 'UV_2_260_mAU'
# We will aggregate them by taking the maximum outlier density across both
  channels
# Local noise threshold: typically defined as mean + k*std of the baseline
  region.
# Let's define peak region as signal > mean_of_baseline + 3*std_of_baseline

# Calculate baseline stats for each group
baseline_stats = df_subset.groupby(['run', 'column'])['UV_1_280_mAU'].agg(['
  mean', 'std']).reset_index()
baseline_stats.columns = ['run', 'column', 'baseline_mean', 'baseline_std']

```



```

# Calculate noise threshold for each group
baseline_stats['noise_threshold'] = baseline_stats['baseline_mean'] + 3 *
    baseline_stats['baseline_std']

# Identify peak region for each channel
peak_region_mask_1 = df_subset['UV_1_280_mAU'] > baseline_stats['
    noise_threshold'].loc[df_subset[['run', 'column']].values]
peak_region_mask_2 = df_subset['UV_2_260_mAU'] > baseline_stats['
    noise_threshold'].loc[df_subset[['run', 'column']].values]

# Calculate outlier density for each channel in peak region
# Peak region: signal > mean_of_peaks + 3*std_of_peaks
peak_signals_1 = df_subset.loc[peak_region_mask_1, 'UV_1_280_mAU']
peak_signals_2 = df_subset.loc[peak_region_mask_2, 'UV_2_260_mAU']

peak_mean_1 = peak_signals_1.groupby(df_subset.loc[peak_region_mask_1, ['run',
    'column']]).mean()
peak_std_1 = peak_signals_1.groupby(df_subset.loc[peak_region_mask_1, ['run',
    'column']]).std()
peak_mean_2 = peak_signals_2.groupby(df_subset.loc[peak_region_mask_2, ['run',
    'column']]).mean()
peak_std_2 = peak_signals_2.groupby(df_subset.loc[peak_region_mask_2, ['run',
    'column']]).std()

# Calculate outlier threshold and density
peak_mean_1 = peak_mean_1.reindex(df_subset[['run', 'column']].unique(),
    fill_value=0)
peak_std_1 = peak_std_1.reindex(df_subset[['run', 'column']].unique(),
    fill_value=0)
peak_mean_2 = peak_mean_2.reindex(df_subset[['run', 'column']].unique(),
    fill_value=0)
peak_std_2 = peak_std_2.reindex(df_subset[['run', 'column']].unique(),
    fill_value=0)

outlier_threshold_1 = peak_mean_1 + 3 * peak_std_1
outlier_threshold_2 = peak_mean_2 + 3 * peak_std_2

# Calculate outlier density for each channel
outlier_density_1 = np.zeros(len(df_subset))
outlier_density_2 = np.zeros(len(df_subset))

# Calculate peak count and outlier count for each group
peak_count_1 = df_subset.loc[peak_region_mask_1, ['run', 'column']].groupby(['
    run', 'column']).size().reset_index(name='peak_count')
peak_count_2 = df_subset.loc[peak_region_mask_2, ['run', 'column']].groupby(['
    run', 'column']).size().reset_index(name='peak_count')

# Merge peak counts back to df_subset
df_subset = df_subset.merge(peak_count_1, on=['run', 'column'], how='left')
df_subset = df_subset.merge(peak_count_2, on=['run', 'column'], how='left')

# Calculate outlier count for each channel
outlier_count_1 = df_subset.loc[df_subset['UV_1_280_mAU'] >
    outlier_threshold_1, ['run', 'column']].groupby(['run', 'column']).size().
    reset_index(name='outlier_count')
outlier_count_2 = df_subset.loc[df_subset['UV_2_260_mAU'] >
    outlier_threshold_2, ['run', 'column']].groupby(['run', 'column']).size().
    reset_index(name='outlier_count')

```

```

# Merge outlier counts back to df_subset
df_subset = df_subset.merge(outlier_count_1, on=['run', 'column'], how='left')
df_subset = df_subset.merge(outlier_count_2, on=['run', 'column'], how='left')

# Calculate outlier density for each channel
df_subset['outlier_density_1'] = df_subset.apply(lambda x: (x['outlier_count_1'] / x['peak_count_1']) * 100 if x['peak_count_1'] > 0 else 0, axis=1)
df_subset['outlier_density_2'] = df_subset.apply(lambda x: (x['outlier_count_2'] / x['peak_count_2']) * 100 if x['peak_count_2'] > 0 else 0, axis=1)

# Aggregate outlier density by taking the maximum across both channels
df_subset['outlier_density'] = df_subset[['outlier_density_1', 'outlier_density_2']].max(axis=1)

# Identify groups with >5% peak outlier density
groups_with_high_outliers = df_subset[df_subset['outlier_density'] > 5][['run', 'column']].drop_duplicates()

# 4. Correlate #Cond_mS_cm# with baseline variance using linear regression slope
# We have baseline_variances per group (run and column). We need to match Cond_mS_cm per group.
# Create a dataframe for regression
group_data = df_subset.groupby(['run', 'column']).agg({
    'Cond_mS_cm': 'first',
    'baseline_var': 'mean'
}).reset_index()

# Filter out NaNs
valid_data = group_data.dropna(subset=['baseline_var', 'Cond_mS_cm'])

if len(valid_data) > 1:
    slope, intercept, r_value, p_value, std_err = stats.linregress(valid_data['Cond_mS_cm'], valid_data['baseline_var'])
    correlation_coefficient = r_value
else:
    slope = np.nan
    correlation_coefficient = np.nan

# Generate Report
report_lines = []
report_lines.append(f"1) Count of zero negative values: {zero_negative_report}")
report_lines.append(f"2) Avg baseline variance per group (Run, Column): {avg_baseline_variance.to_dict()}")
report_lines.append(f"3) Groups with >5% peak outlier density: {groups_with_high_outliers.to_dict('list')}")
report_lines.append(f"4) Correlation coefficient (Cond_mS_cm vs Variance): {correlation_coefficient:.4f}")
report_lines.append(f"    Slope: {slope:.4f}")

report_text = "\n".join(report_lines)

# Save report to /workspace/
with open('/workspace/quality_stratification_report.txt', 'w') as f:
    f.write(report_text)

print(report_text)
###

```

Listing 3: Code_3